

DataMind

AI-Powered Multi-Agent Data Ingestion & Natural-Language Analytics Platform

Prepared for: Hackathon submission / project reference

Scope: architecture, request-flow, feature status, setup

Grounded in the running codebase – not a proposal

DataMind takes raw business data — a CSV, an Excel file, a JSON export — and turns it into a dataset a non-technical person can question in plain English. An AI classification agent figures out what the data is (orders? customers? deliveries?) and proposes a Snowflake schema; a human reviews and confirms it before anything is written. From there, every question asked in the dashboard is understood, translated into validated read-only SQL, executed against that user's own isolated Snowflake schema, and turned back into a plain-English answer — complete with timelines, summaries, and honest gaps when the data doesn't have what was asked.

01 Overview & problem

Most internal analytics tools ask the user to already know their schema: which table, which join, which column holds the answer. DataMind removes that requirement in both directions. On the way in, an AI agent inspects raw, messy column headers (prod_id, qty, wh) and proposes a clean, typed Snowflake schema — but never writes it without a human confirming the mapping first. On the way out, a user asks a question the way they'd ask a colleague — "what happened with order ORD100000?", "what was our total revenue last month?" — and the system figures out which tables that spans, writes the SQL, and explains the result in prose, a timeline, or a chart, whichever fits.

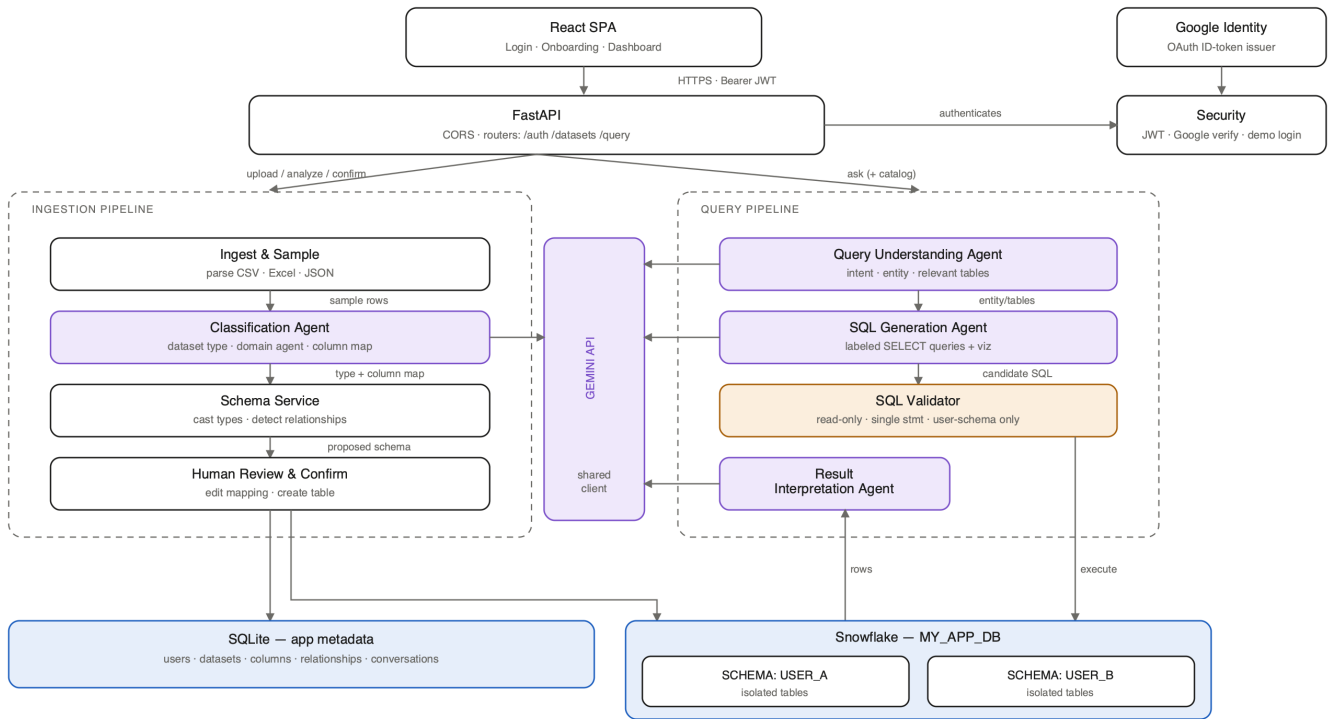
Two design decisions shape everything downstream: **the system is multi-agent, not one giant prompt** (a separate, focused Gemini call for classification, for query understanding, for SQL generation, and for result interpretation), and **every user's data lives in its own Snowflake schema**, named from a hash of their internal id, never a shared table filtered by a user_id column. Isolation is enforced by what SQL is allowed to reference, not by application-level trust.

02 Tech stack

Layer	Choice	Notes
Frontend	React + TypeScript + Vite	Apple-HIG-inspired design system, Recharts for visualization
Backend	FastAPI (Python 3.12)	Routers: /auth, /datasets, /query
App metadata	SQLite + SQLAlchemy	Users, datasets, columns, relationships, conversation history
Analytical store	Snowflake	One schema per user; created and loaded via snowflake-connector-python
AI	Google Gemini (gemini-3.5-flash-lite)	One shared client; rate-limit retry with backoff
Auth	Google Sign-In (OAuth) + Demo login	JWT issued after either; demo login bypasses OAuth for testing two isolated accounts

03 System architecture

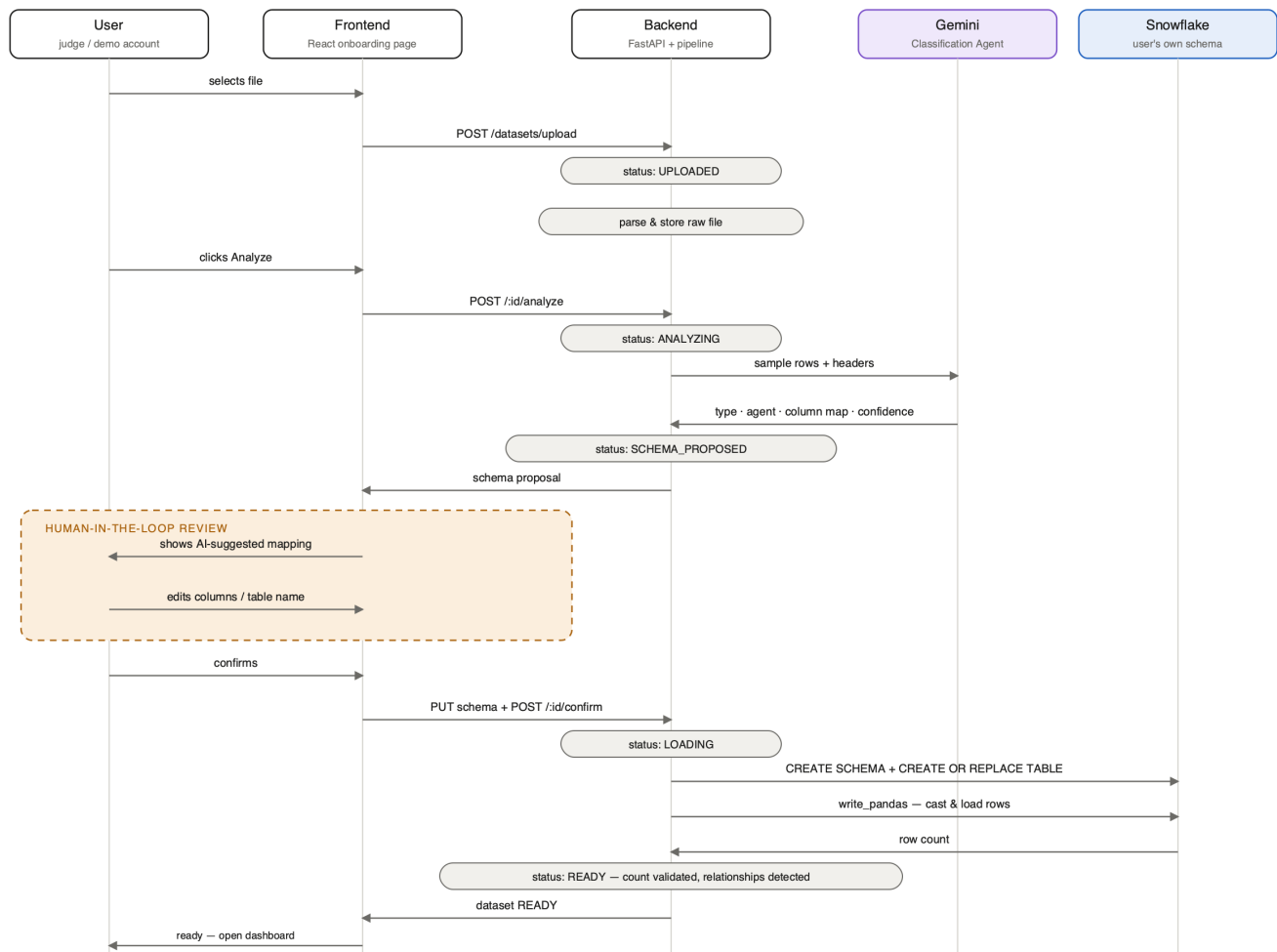
One FastAPI backend fronts two agent pipelines that share a single rate-limited Gemini client. Every dataset a user loads lands in that user's own Snowflake schema — never a shared table filtered by a user_id column, an actual separate schema.



■ **What this shows** — a request enters through FastAPI, which fans out to whichever pipeline the route needs. Every AI step on both sides — classification, query understanding, SQL generation, result interpretation — calls the **same** Gemini client, which is why one rate-limit retry helper protects the whole system, not just one agent. The two boxes inside Snowflake are not a metaphor: they are two actual schemas, named USER_<sha256 hash>, never the raw account id. No arrow connects them, because no query ever does either — the SQL Validator rejects any generated statement that references a schema other than the requesting user's own.

04 Workflow — data onboarding

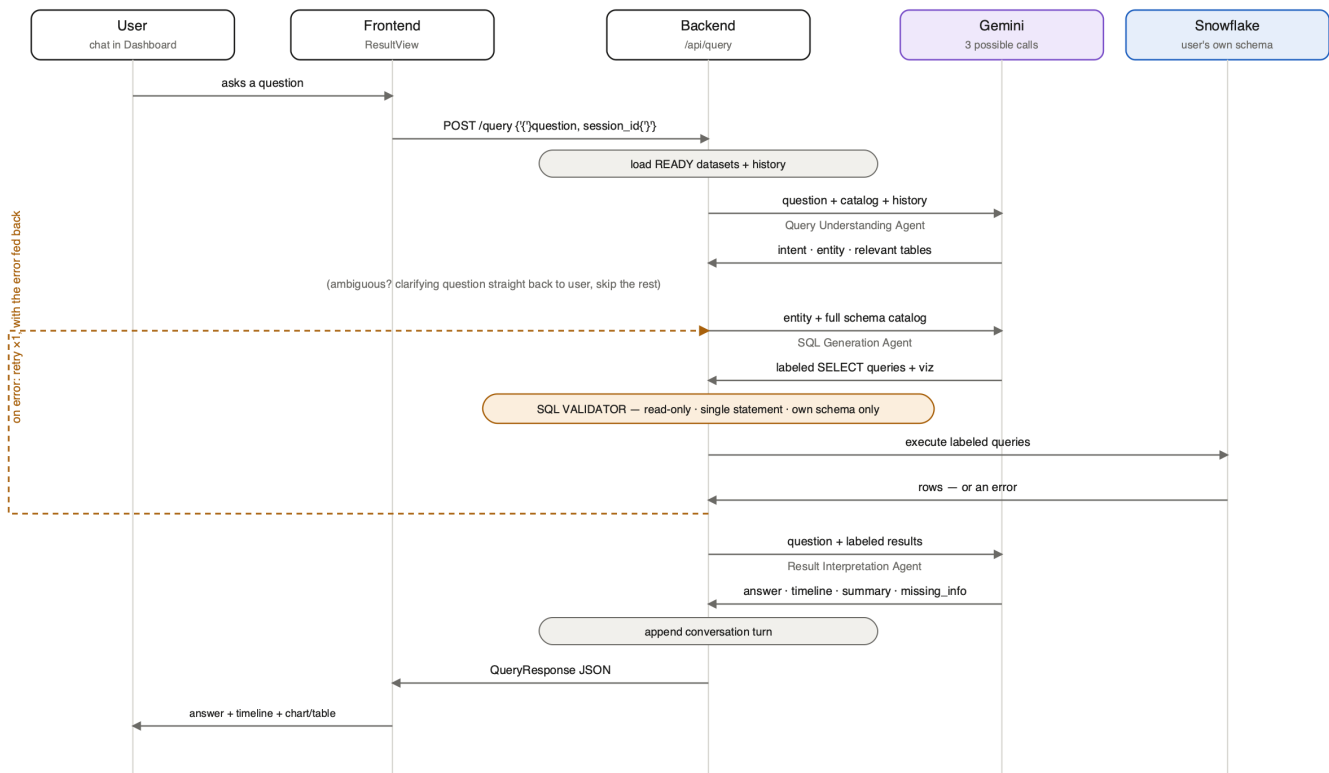
From a dropped CSV to a queryable Snowflake table. The dataset's status moves through five states — **UPLOADED** → **ANALYZING** → **SCHEMA_PROPOSED** → **LOADING** → **READY** (or **FAILED** at any step) — and the pipeline deliberately stalls at **SCHEMA_PROPOSED** until a person confirms the mapping.



What this shows — the pipeline never touches Snowflake before a human sees and can edit the AI's column mapping; that pause is the **SCHEMA_PROPOSED** state, not a UI convenience. Confirming runs `CREATE OR REPLACE TABLE` in the user's own schema, loads the cast dataframe with `write_pandas`, then re-queries Snowflake for the actual row count rather than trusting the load call's return value, before attempting relationship detection against the user's other **READY** datasets.

05 Workflow — natural-language query

One chat message costs three Gemini calls — understanding, SQL generation, interpretation — unless the question is ambiguous, in which case it short-circuits after the first. A failed query gets exactly one retry, fed the real Snowflake error.



What this shows — the SQL Validator is a hard gate, not a suggestion: it parses the generated SQL and rejects anything that isn't a single read-only SELECT/WITH statement scoped to tables in the requesting user's own schema, before Snowflake ever sees it. If Snowflake still rejects the (validator-approved) query — a bad column name, a type mismatch — the backend sends that exact error back to the SQL Generation Agent once and tries again; a second failure returns a plain "couldn't answer that" instead of looping forever. The Result Interpretation Agent only ever sees the rows that came back, which is what makes `missing_info` reliable: it can't invent a delivery ETA that was never in the result set.

06 Features implemented

- AI-driven dataset classification, routed to one of six specialized domain agents (inventory, customer, financial, order, delivery, product)
- Human-in-the-loop schema review — every AI-suggested column mapping, type, and table name can be edited before anything is written to Snowflake
- Per-user Snowflake schema isolation, enforced by a SQL validator (read-only, single-statement, own-schema-only), not just by naming convention
- Automatic cross-dataset relationship detection (foreign-key columns matched to other datasets' primary keys, by exact name first, fuzzy entity match as fallback)
- Natural-language analytics questions ("what was our total revenue last month?") with automatic chart/table/KPI selection
- Entity investigation across multiple tables from a single identifier — including one that only exists in a secondary table (e.g. a payment's transaction id resolves through to its order's delivery status)
- Conversational follow-ups within a session, using recent chat history to resolve "that order", "the previous month"
- One retry with real error feedback when generated SQL fails execution, before giving up gracefully
- A one-click demo data seeder: 5 relationally-linked datasets (~5,000 rows each) through the full real pipeline, for fast end-to-end testing

07 Setup & running locally

Backend

```
cd backend && python3.12 -m venv venv && source venv/bin/activate && pip install -r requirements.txt
```

Copy `.env.example` to `.env` and fill in: a JWT secret, a Google OAuth client id, a Gemini API key, and Snowflake account/user/password (or key-pair)/role/warehouse/database.

Run: `uvicorn app.main:app --reload --port 8001`

Frontend

```
cd frontend && npm install
```

Copy `.env.example` to `.env`; set `VITE_API_BASE_URL` to the backend's URL and `VITE_GOOGLE_CLIENT_ID` to match the backend's.

Run: `npm run dev`

Fastest path to a working demo

Sign in with either Demo User A or Demo User B (bypasses Google OAuth entirely), then on the Onboarding page click **Load demo datasets** — this seeds 5 real, related datasets through the full pipeline in about a minute, ready to query from the Dashboard.

08 Current status & known limitations

Verified end-to-end against a live Gemini API key and a live Snowflake account: ingestion, classification, schema confirmation, table creation and load, relationship detection, per-user isolation, analytics queries, and multi-table entity investigation.

Open items

- Real Google Sign-In is gated behind an OAuth consent-screen restriction on the Google Cloud project (org-internal / test-user allowlist) — Demo Login is the working path today.
- Gemini's free tier caps at a small number of requests per day per model; a live demo with several questions asked back-to-back should run on a billing-enabled key.
- The manual upload → review → edit-mapping → confirm flow, Excel/JSON upload, and multi-turn follow-up questions are implemented but have had lighter end-to-end testing than the demo-seed path.
- No automated test suite yet; verification so far has been direct, manual, against the real stack.